

ChildTaskTracker

A static, backend-free front end: every screen, every write, and the optional AI call all happen inside one browser tab. The three figures below show how a file on GitHub becomes a running app, how a tap becomes persisted state, and where AI sits — as an optional, guarded detour, never a dependency.

v1.0 2026-09-16 github.com/adamhelwa/ChildTaskTracker

1 • Deployment

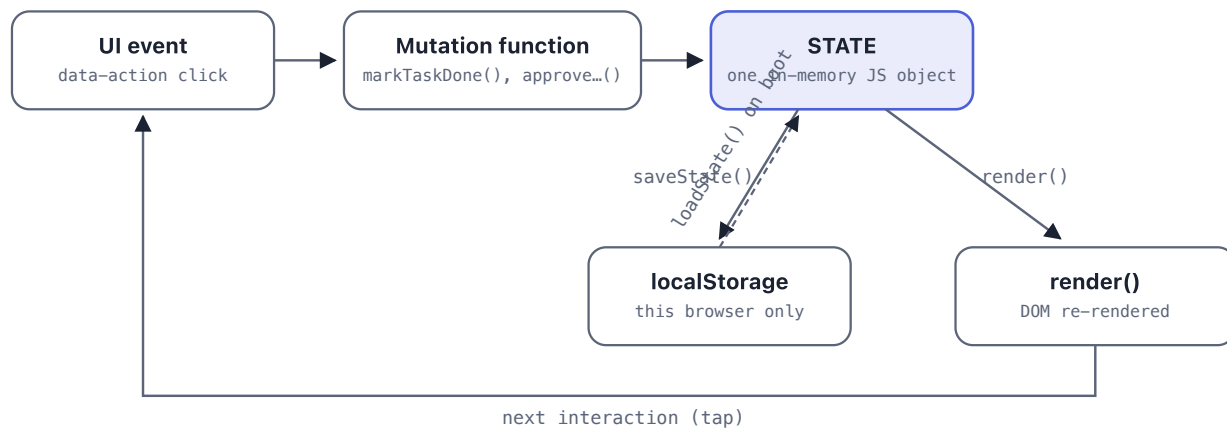
How the four files in the repo reach a browser — no build step, no server.



What this shows: there is no server between the repo and the browser — the deployed artifact *is* the four static files, fetched fresh on every load and re-fetched on every deploy via a versioned query string.

2 • Local data loop

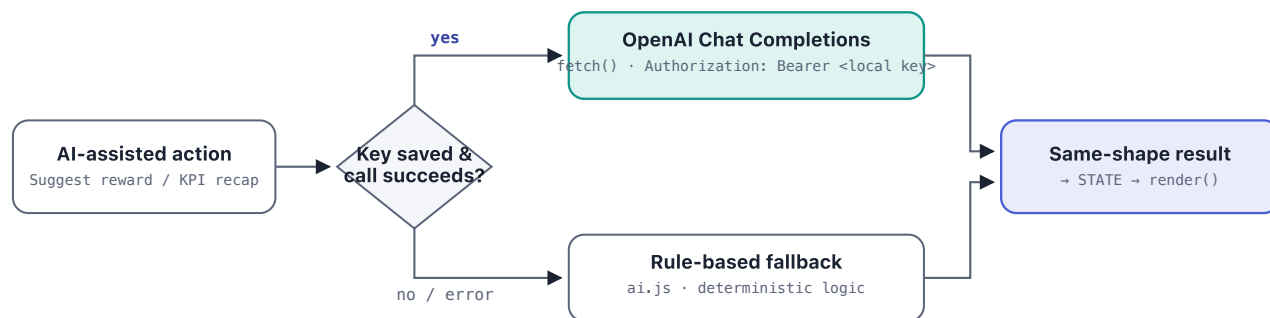
What happens inside the browser for every tap — marking a task, approving it, redeeming a reward, editing a name.



What this shows: every action follows the same one-way loop, and **localStorage is the only durable copy** — nothing here talks to a server, so this device is the single source of truth for this family's data.

3 • AI request path

What happens when a parent taps an AI-assisted button — always resolves to the same shape of result, with or without a key.



What this shows: the browser talks to OpenAI directly — there is no proxy to hide the key behind — but the app never *depends* on that call succeeding; a missing key, a network error, and a 429 all resolve through the same fallback lane to the same result shape.

Tradeoffs, insights, and risk management

The decisions the three figures above are actually making, and why.



Design tradeoffs

No backend, no cross-device sync

Static hosting means zero infrastructure and "the HTML is the deploy," but each browser's localStorage is its own island — a parent's phone and a child's tablet don't sync. Two-parent sharing is a view switcher, not real accounts.

Client calls OpenAI directly

No proxy to hide or rate-limit the key server-side — traded for matching the "no backend" brief exactly, and for keeping every user's key local to their own browser, never in the repo.

Vanilla JS over a framework

No component diffing — the view re-renders from an HTML template string on every change. Traded for zero build step: the four files in the repo are the entire deployable artifact.

Fallback as a first-class feature

Every AI-backed feature ships two implementations, model and rule-based, doubling the code per feature so the product behaves identically with or without a key.



Technical insights

One stopPropagation broke every modal

Delegating all clicks to a single document listener means anything that stops a click from bubbling silently disables the app below that point — only live click-testing caught it, not reading the code.

Hash routing vs. headless navigation

The router relies on real `location.hash` assignment, which fires `hashchange`; a testing tool's `pushState`-style navigation doesn't, and looked like a bug in the app when it wasn't.

The CDN hides your own fixes

GitHub Pages' edge cache served stale JS after a push even once the origin was confirmed correct; versioned query strings made re-verification deterministic.

Escaping has to cover AI output too

An OpenAI-suggested emoji or title is exactly as untrusted as anything a user typed — both land in the same `innerHTML` template.



Risk management

Data loss

A family's entire history lives in one browser with no backup. Mitigated with an explicit, confirmed "Reset demo data" action rather than a silent clear, and documented as a known limitation.

API key exposure

The key never leaves the browser it was entered in and is never written to a file this repo publishes — cloning the repo can never leak a key.

Unsafe AI output reaching a child

Reward suggestions are constrained to the parent's own catalog by prompt design, money/screen-time are explicitly disallowed, and every call falls back to the same result shape so a bad response never surfaces as broken UI.

XSS from user or AI text

One `esc()` helper escapes every dynamic string before it's templated into the DOM, applied uniformly to human-entered and AI-generated strings alike.